

Operator-supplied material – "game changers" (–)

Source: third-party AI transcript supplied by the operator as research material.

STATUS: UNVERIFIED CLAIMS. Material #1 from the same source proved ~2/3 accurate, ~1/3 confabulated (fabricated CLI commands, a deprecated package, a package whose source repo is deleted, and a hook example that would silently never block). Treat accordingly.

Operator implementation constraint (binding): any codebase we write to incorporate this MUST be **bash scripts and Go** (Gin gonic for networking). The material's Python/JS implementations are therefore reference-only, never adoption targets.

GC – Mutation Testing Gate ("kill fake tests")

- Claim: AI writes tests that pass on trivial implementations; mutation testing flips this.
- Mechanism: inject mutants (`>` → `>=` , drop `!` , `+` → `-`); tests must KILL 100% of mutants.
- **Hard rule claimed: ZERO surviving mutants; feature not complete until 100% kill.**
- Named tooling: `stryker-js` (JS/TS), `mutmut` (Python), thresholds `break: 100` .
- Integration claimed: PreToolUse hook running stryker on `git commit` ; a `fix_mutants.sh` loop feeding surviving mutants back to `claude --print` .

GC- – Adversarial "Saboteur" agent (red-team loop)

- Claim: a dedicated agent whose sole KPI is BREAKING the builder's work, run BEFORE "done".
- Mechanism: fuzzing, race conditions, state corruption, resource exhaustion, dependency failure injection. Gate: 100+ disruptive cases with no crash; any crash → builder restarts loop.
- **Claimed success criterion: the Saboteur has FAILED if it finds 0 failures ("try harder").**
- Named: `.claude/agents/Saboteur.md` , `adversarial_runner.py` fuzz harness, "CodeHacker framework".

GC- – Semantic contract checking via vector RAG ("Achilles heel scan")

- Claim: code can be syntactically fine but violate implicit architectural contracts.
- Mechanism: vectorise new code's call graph/imports/data flow; query a RAG DB of ADRs and known anti-patterns; **block if cosine similarity > 0.85 (elsewhere stated as distance < 0.3).**
- Named: `chromadb` , `sentence-transformers` (`all-MiniLM-L6-v2`), `langchain` , `archgate` (dotnet tool).

GC- – Progressive Atomic Delivery ("Zeno compiler")

- Claim: force sub-50-line shippable units; each must compile in isolation, pass tests in a clean sandbox, and do one real CRUD against a live staging DB via MCP.
- Gate: if unit N fails, the AI may not start unit N+1; session halts.
- **Claimed effect: "cuts false success by over 90% in internal trials"** (no citation given).
- Also claimed: block commits >5 files, >50 lines per file.

GC- — Live telemetry shadowing (production reality check)

- Claim: deploy candidate to a shadow env, mirror 1% of real production traffic, diff responses.
- Gate: 10,000 shadow requests over 24h at <0.1% deviation before QA.
- Named: Istio `mirror` / `mirrorPercent` , k8s shadow deployment, an MCP server exposing `get_shadow_report` / `get_shadow_failures` to the agent.

Unified pipeline claimed

Plan (GC-3) → Build (GC-4) → Self-test (GC-1) → Harden (GC-2) → Stage (GC-5) → `[VERIFIED-REAL]` . Claimed: "the system literally cannot produce a false green light". Setup shown as `git clone https://github.com/your-org/zero-bluff-gates.git` — a PLACEHOLDER, not a real repo.

Conductor's pre-flagged concerns (verify, do not assume)

1. The `$CLAUDE_TOOL_INPUT` hook pattern appears **AGAIN** (GC-1's PreToolUse). Material #1's identical pattern was **PROVEN** to silently never block — a fabricated guard that is itself the §11.4.201 false-negative shape. Assume the same until re-proven.
2. **"100% mutation kill" contradicts measured industry practice**. Our own Phase 2 research found Google runs mutation testing **diff-scoped at review, never whole-corpus** (TSE'22). A 100% whole-corpus threshold is plausibly unachievable and gaming-inducing. Resolve this.
3. **Cosine-similarity as a blocking gate (GC-3)** is a §11.4.201 false-positive generator by construction; arbitrary thresholds (0.85 / 0.3) with no calibration. A gate that misfires loses its trust budget (Phase 2: measured 90% override rates on mis-calibrated fleets).

4. **The 50-line atomic limit (GC-4)** is an arbitrary magic number presented as a law, and the ">90% reduction in internal trials" claim carries no citation. Both need evidence.
5. **GC-5 may be largely INAPPLICABLE to our domain.** We ship FIRMWARE to physical devices; there is no stream of production HTTP requests to mirror. Ask instead: what is the firmware analogue of shadow traffic, and does one exist that is not a fantasy?
6. **GC-1 and GC-2 map onto things we ALREADY have** — §1.1 paired mutations (hand-authored per gate) and §11.4.85 stress+chaos. The question is whether automated/whole-corpus mutation is a genuine upgrade over hand-authored per-gate mutation, and whether an adversarial agent binds where §11.4.85 currently prescribes.